L26 — Recursion: Principles, Importance, and Implementation

Unit: 2 | **Chapter:** Stacks & Recursion | **BT Level:** BT5 | **CO:** CO2, CO3

Learning Objectives

- Define recursion and identify the base case and recursive case
- Trace recursive calls using the call stack
- Implement classic recursive algorithms
- Convert between recursive and iterative solutions
- Analyze time and space complexity of recursive algorithms

---

1. What is Recursion?

Recursion is a technique where a function calls itself to solve a smaller version of the same problem, until it reaches a **base case** that can be solved directly.

Two essential components:

1. **Base case** — the simplest input, solved without further recursion
2. **Recursive case** — breaks problem into smaller sub-problem and calls itself

> "To understand recursion, you must first understand recursion."

---

2. How Recursion Uses the Call Stack

Every function call creates a **stack frame** containing:

- Function parameters
- Local variables
- Return address

```
def factorial(n):
    if n == 0:            # Base case
        return 1
    return n * factorial(n - 1)   # Recursive case

factorial(4)
```

Call stack visualization:

```
factorial(4) calls factorial(3) calls factorial(2) calls factorial(1) c

Stack grows:                    Stack unwinds:
factorial(4) → waiting          factorial(0) = 1
```

```
factorial(3) → waiting          factorial(1) = 1*1 = 1
factorial(2) → waiting          factorial(2) = 2*1 = 2
factorial(1) → waiting          factorial(3) = 3*2 = 6
factorial(0) = 1                factorial(4) = 4*6 = 24
```

Maximum stack depth = n → O(n) space.

---

3. Classic Recursive Algorithms

3.1 Factorial

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial undefined for negative numbers")
    if n == 0:       # Base case
        return 1
    return n * factorial(n - 1)   # T(n) = T(n-1) + O(1) → O(n)
```

3.2 Fibonacci

```
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)
# T(n) = T(n-1) + T(n-2) + O(1) → O(φⁿ) where φ ≈ 1.618
# Exponential! Many subproblems recomputed.
```

With memoization (top-down DP):

```
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
# T(n) = O(n)
```

3.3 Power Function

```
def power(base, exp):
    if exp == 0:          # Base case
        return 1
    if exp % 2 == 0:      # Even exponent: base^exp = (base^(exp/2))²
        half = power(base, exp // 2)
        return half * half
    return base * power(base, exp - 1)   # Odd exponent
# T(n) = T(n/2) + O(1) → O(log n)  [fast exponentiation]
```

3.4 Binary Search (Recursive)

```
def binary_search(arr, low, high, target):
    if low > high:
        return -1
    mid = low + (high - low) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, mid + 1, high, target)
    else:
        return binary_search(arr, low, mid - 1, target)
# T(n) = T(n/2) + O(1) → O(log n)
```

3.5 Tower of Hanoi

Move n disks from peg A to peg C using peg B as auxiliary:

```
def hanoi(n, source, destination, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return
    hanoi(n - 1, source, auxiliary, destination)   # Move n-1 from A to
    print(f"Move disk {n} from {source} to {destination}")  # Move disk
    hanoi(n - 1, auxiliary, destination, source)  # Move n-1 from B to

hanoi(3, 'A', 'C', 'B')
# T(n) = 2T(n-1) + O(1) → T(n) = 2ⁿ - 1 → O(2ⁿ)
```

---

4. Recursion Tree and Complexity Analysis

Factorial

$$T(n) = T(n-1) + c \Rightarrow T(n) = nc = O(n)$$

Binary Search

$$T(n) = T(n/2) + c \Rightarrow T(n) = c\log_2 n = O(\log n)$$

Merge Sort

$$T(n) = 2T(n/2) + cn \Rightarrow T(n) = O(n\log n)$$

Fibonacci (naive)

$$T(n) = T(n-1) + T(n-2) + c \Rightarrow T(n) = O(\varphi^n)$$

---

5. Recursion vs Iteration

Aspect	Recursion	Iteration
Code clarity	Often cleaner for divide-conquer	More verbose for recursive problems

Stack space	O(depth) — can overflow	O(1)
Performance	Function call overhead	Generally faster
Expressiveness	Natural for trees, graphs, D&C	Natural for loops
Stack overflow risk	Yes (deep recursion)	No

Converting Recursion to Iteration (using explicit stack)

```
# Recursive factorial
def factorial_recursive(n):
    if n == 0: return 1
    return n * factorial_recursive(n-1)

# Iterative equivalent
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Recursive DFS → Iterative with explicit stack
def dfs_iterative(graph, start):
    stack = [start]
    visited = set()
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                stack.append(neighbor)
```

---

6. Tail Recursion

A **tail recursive** function makes its recursive call as the **last action** — no computation after the call.

```
# NOT tail recursive: multiplication after recursive call
def factorial(n):
    if n == 0: return 1
    return n * factorial(n-1)   # Multiply happens AFTER call returns

# Tail recursive: accumulator carries the state
def factorial_tail(n, acc=1):
    if n == 0: return acc
    return factorial_tail(n-1, n * acc)   # Call is last action
```

Tail recursive functions can be optimized by compilers to use O(1) stack space (tail call optimization — TCO). Python does **not** perform TCO natively.

---

7. Pitfalls

Pitfall	Description	Fix
Missing base case	Infinite recursion → stack overflow	Always define base case
Wrong base case	Incorrect results	Verify smallest input manually
No progress toward base	Infinite recursion	Ensure n decreases
Deep recursion in Python	`RecursionError` at depth ~1000	Use `sys.setrecursionlimit` or iterate

```
import sys
sys.setrecursionlimit(10000)  # Increase limit (use with caution)
```

Summary

- Recursion = function calling itself with smaller input; needs a **base case** and **recursive case**.
- Each recursive call uses O(1) stack frame → total stack space = O(call depth).
- Naive Fibonacci is $O(2^n)$; memoized is O(n).
- **Tail recursion** can be converted to iteration with O(1) space.
- For Python: prefer iteration for deep recursion (avoids stack overflow).

References

- Cormen et al., *Introduction to Algorithms*, Ch. 2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 7 (Springer, 2020)